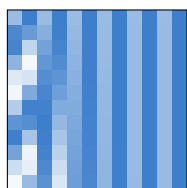


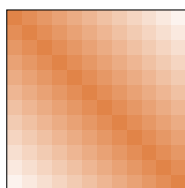
Position Encoding Methods and Length Generalization

Introduction to Large Language Models — Week 12

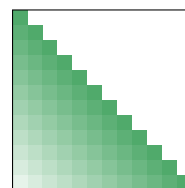
Detailed study notes based on the lecture slides of Prof. Mitesh M. Khapra
(AI4Bharat, Dept. of CSE, IIT Madras)



Sinusoidal APE



Relative $j - i$ RoPE rotation



ALiBi bias

Structure: Recap & taxonomy $\rightarrow$ Module 1 (limits of absolute PE) $\rightarrow$ Module 2 (relative PE: Shaw, Transformer-XL, T5, RoPE) $\rightarrow$ Module 3 (length generalization: ALiBi, NoPE) $\rightarrow$ Practice set

Contents

How to use these notes	3
1 Recap: absolute positional encoding and the design space	4
1.1 Why positions at all?	4
1.2 Absolute positional encoding (APE)	4
1.3 Question everything	5
1.4 Taxonomy of position encodings	6
2 Module 1: Limitations of absolute position encoding	7
2.1 Learned APE is everywhere	7
2.2 Does APE learn relative positions? The shift experiment	7
2.3 Limitation 2: no extrapolation beyond the context length	8
2.4 Limitation 3: length generalization (within the context)	8
3 Module 2: Relative position encoding methods	10
3.1 One absolute position vs T relative positions	10
3.2 The naive implementation is expensive	11
3.3 The idea: decompose the pre-attention score	11
3.4 Shaw et al. (2018): relative position embeddings in attention	12
3.4.1 Clipping	12
3.4.2 The full Shaw attention	13
3.5 RPE variations	14
3.5.1 Transformer-XL (recursively extends context)	14
3.5.2 T5 bias	14
3.6 Rotary Position Embedding (RoPE)	16
3.6.1 A unifying view	16
3.6.2 The problem with adding: APE changes angles	16
3.6.3 Derivation with complex numbers	17
3.6.4 The same thing as a rotation matrix	17
3.6.5 Extending to $d > 2$ dimensions	18
3.6.6 Properties and performance	19
4 Module 3: Position encoding and length generalization	21
4.1 The question	21
4.2 The extrapolation experiment (Press et al., ALiBi paper)	21
4.3 Attention with Linear Biases (ALiBi)	21
4.3.1 ALiBi acts like local windowed attention	22
4.3.2 Beyond ALiBi	23

4.4	Is that all? No position encoding (NoPE)	23
4.5	Is there an inherent limitation?	24
4.6	Summary of position encodings	25
5	Consolidated summary and formula sheet	26
5.1	Concept map	26
5.2	Formula sheet	26
5.3	Comparison at a glance	27
5.4	Ten sentences to remember	27
6	Practice set	28
6.1	Part A: Numerical problems	28
6.2	Part A: Solutions	29
6.3	Part B: Subjective questions	31
6.4	Part B: Model answers	31
6.5	Part C: Multiple-choice questions	34
6.6	Part C: Answer key with explanations	35

How to use these notes

Each module follows the same pattern: the *problem*, the *intuition*, the *mathematics* (with matrix shapes written underneath symbols, $X_{T \times d}$), a *diagram*, and a summary. Box colours: **Intuition** (blue), **Key idea** (green), **Derivation** (grey), **Note** (orange: extra context or clarifications of the slides), **Common pitfall** (red), **Worked example** (purple).

Notation

Symbol	Meaning
T	context length (maximum sequence length used in training)
i, j (or m, n)	positions of the query token and key token (m, n are used in RoPE to avoid clashing with the imaginary unit)
$x_i \in \mathbb{R}^{d_{\text{model}}}$	word (token) embedding of the token at position i
$p_i \in \mathbb{R}^{d_{\text{model}}}$	absolute position embedding (APE) of position i
$h_i = x_i + p_i$	input to the first layer when APE is used
d	per-head dimension ($d = d_{\text{model}}/n_h$)
W_Q, W_K, W_V	query/key/value projections
q_i, k_j	query and key vectors; $e_{ij} = q_i k_j^\top$ is the <i>pre-attention score</i> (logit)
α_{ij}	attention weight = $\text{softmax}_j(e_{ij})$
$j - i$	relative position (distance) of key j as seen from query i
p_{ij}^K, p_{ij}^V	relative position embeddings added to keys / values (Shaw et al.)
k	clipping distance for relative positions
r_{j-i}	scalar relative bias (T5)
$R_{\Theta, m}^d$	RoPE rotation matrix for position m
m (ALiBi)	head-specific slope
T_{valid}	sequence length used at inference/validation time

1 Recap: absolute positional encoding and the design space

1.1 Why positions at all?

Self-attention is **permutation-equivariant**: if you shuffle the input tokens, the outputs are shuffled in exactly the same way, and each token's output is unchanged. Attention sees a *set*, not a sequence. “dog bites man” and “man bites dog” would get identical token representations. Positional encoding (PE) is how order is injected.

Derivation: attention ignores order without PE

Let P be a permutation matrix and $X' = PX$ the shuffled input. Then $Q' = PXW_Q = PQ$, $K' = PK$, $V' = PV$ and

$$\text{softmax}\left(\frac{Q'K'^\top}{\sqrt{d}}\right)V' = \text{softmax}\left(\frac{PQK^\top P^\top}{\sqrt{d}}\right)PV = P \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)P^\top PV = PO,$$

because a row-wise softmax commutes with permuting rows and columns, and $P^\top P = I$. The output is just the old output permuted: no information about order was used.

1.2 Absolute positional encoding (APE)

The vanilla Transformer encodes the **absolute** position of each token and **adds** a position vector to the word embedding:

$$\begin{matrix} h_i & = & x_i & + & p_i \\ 1 \times 512 & & 1 \times 512 & & 1 \times 512 \end{matrix}$$

The position vectors are **fixed** (not learned), given by sinusoids:

$$PE(i, j) = \begin{cases} \sin\left(\frac{i}{10000^{2j/d_{\text{model}}}}\right), & j = 0, 2, \dots \\ \cos\left(\frac{i}{10000^{2j/d_{\text{model}}}}\right), & j = 1, 3, \dots \end{cases}$$

where i is the position (row) and j indexes the embedding dimension (column).

Note: the standard indexing

The slide formula is a compact form. The original paper pairs dimensions: for $k = 0, 1, \dots, d_{\text{model}}/2 - 1$,

$$PE(i, 2k) = \sin(i\omega_k), \quad PE(i, 2k+1) = \cos(i\omega_k), \quad \omega_k = 10000^{-2k/d_{\text{model}}}.$$

Dimensions $2k$ and $2k+1$ share the *same* frequency ω_k : one gets sine, the other cosine. The frequencies fall geometrically from $\omega_0 = 1$ (wavelength 2π) to about $1/10000$ (wavelength $\approx 2\pi \cdot 10^4$). Keep this pairing in mind; RoPE reuses exactly these frequencies.

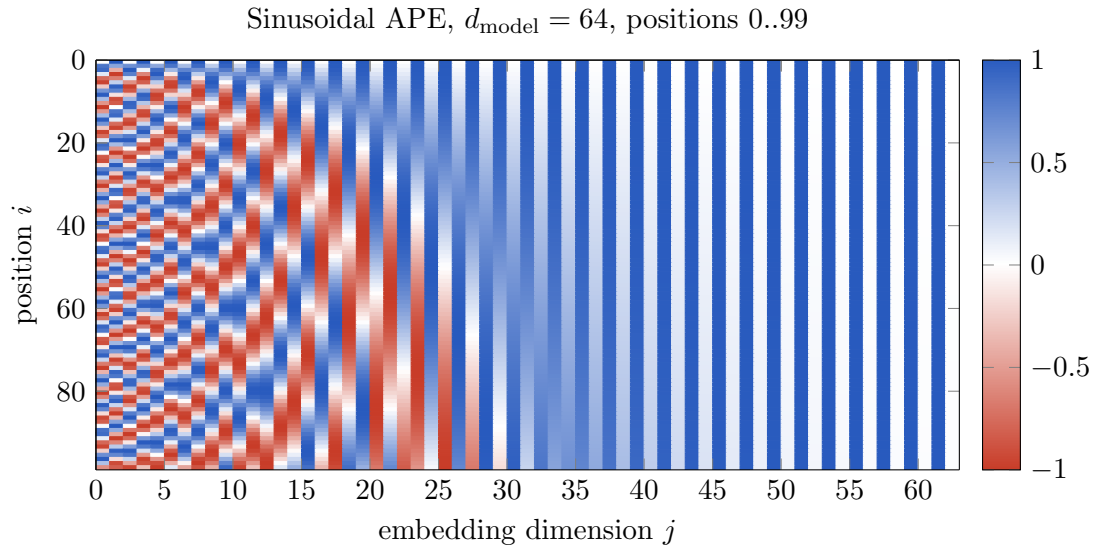


Figure 1: Each row is one position vector p_i . Low dimensions (left) oscillate quickly with position; high dimensions (right) change slowly. Every position gets a unique “barcode”.

Intuition: a binary clock made of waves

Think of counting in binary: the last bit flips every step, the next every 2 steps, the next every 4, and so on. Sinusoidal PE is a smooth version of this. Fast waves distinguish neighbouring positions, slow waves distinguish far-apart ones. Together they give every position a unique, bounded $([-1, 1])$ code.

Derivation: shifting by k is a rotation (the seed of RoPE)

For one frequency pair $(\sin i\omega, \cos i\omega)$ and a shift k :

$$\begin{bmatrix} \sin((i+k)\omega) \\ \cos((i+k)\omega) \end{bmatrix} = \begin{bmatrix} \cos k\omega & \sin k\omega \\ -\sin k\omega & \cos k\omega \end{bmatrix} \begin{bmatrix} \sin i\omega \\ \cos i\omega \end{bmatrix}.$$

So p_{i+k} is a *fixed linear function* (a rotation that depends only on k) of p_i . The original authors hoped this would let the model learn to attend by relative position, and even extrapolate to longer sequences. As Module 3 shows, that hope did not really work out for additive APE. RoPE (Module 2) makes this rotation property do the actual work.

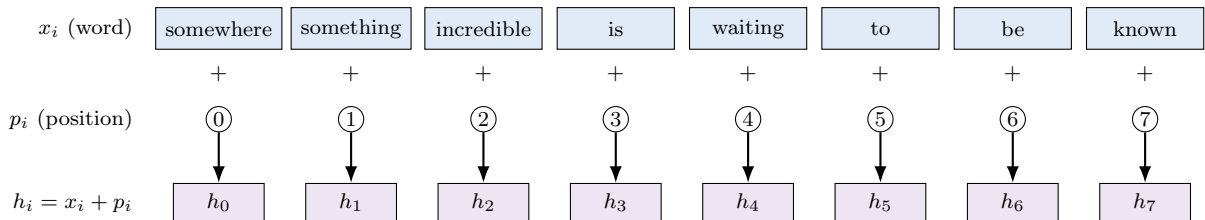


Figure 2: APE: each token gets exactly one position index $0, 1, \dots, T-1$, and its vector is added to the word embedding once, at the input.

1.3 Question everything

Every design choice in APE can be challenged:

Design choice in vanilla APE	Alternative	Where it leads
encode absolute positions (0, 1, 2, 3)	relative positions (−2, −1, 0, 1)	Shaw, T5, Transformer-XL, RoPE, ALiBi
add p_i to x_i : $x_i + p_i$	multiply : $x_i \odot f(p_i)$	RoPE (rotation is multiplicative), Complex embeddings
inject at the input	add to the attention score : $q_i k_j^\top + f(p_i, p_j)$	T5 bias, ALiBi
values are fixed (sin/cos)	learn them: $PE(i, j) = \theta$, trained by ∇	BERT, GPT, BART, OPT (learned APE)

This is still an active research area (alongside attention mechanisms themselves).

1.4 Taxonomy of position encodings

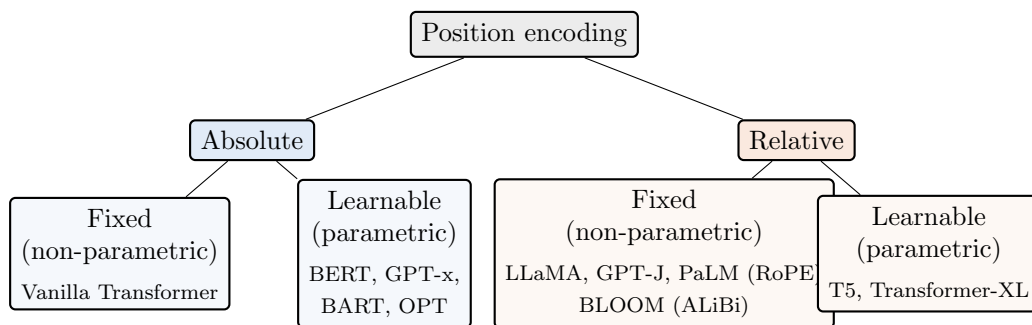


Figure 3: Two axes: *what* is encoded (absolute vs relative) and *how* (fixed formula vs learned parameters).

Note: the slide’s grouping

The slide lists LLaMA, GPT-J, PaLM and BLOOM under “Relative, fixed”. LLaMA, GPT-J and PaLM use RoPE; BLOOM uses ALiBi. Both are fixed formulas that end up depending only on relative distance. Transformer-XL is listed as learnable because, although its R_{j-i} is sinusoidal, it adds the learnable vectors u, v (and a learned projection of R).

2 Module 1: Limitations of absolute position encoding

2.1 Learned APE is everywhere

All the prominent pre-trained models (BERT, RoBERTa, GPT-x, BART, OPT) use **learned** APE: a trainable lookup table with one row per position,

$${}_{T \times d_{\text{model}}} P = \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_{T-1} \end{bmatrix}, \quad h_i = x_i + P[i].$$

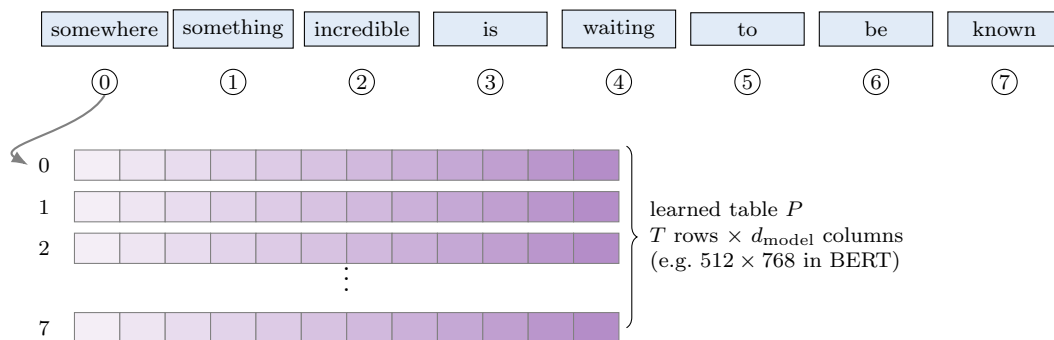


Figure 4: Learned APE: position i simply selects row i of a trainable matrix. Nothing ties row 3 to row 4. They are independent parameters.

2.2 Does APE learn relative positions? The shift experiment

It is **hypothesised** that models with APE still learn to attend to tokens by their *relative* position (the meaning of “the word two to the left” should not depend on where the sentence starts). Why does this matter? Because language is largely shift-invariant, and because relative reasoning is what lets a model generalise to new lengths. **Does it actually happen?**

Experiment (Sinha et al., 2022). Take a pre-trained model (e.g. GPT-2) and use it for a **grammatical acceptability** test based on perplexity. We expect the prediction for a sentence to be consistent. Now **shift the starting position** by k : the first token gets position $0 + k$ instead of 0.

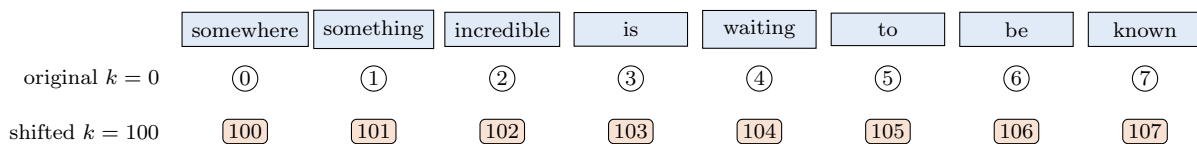


Figure 5: Shifting all positions by k leaves every *relative* distance unchanged.

If the model had truly learned relative positions, the predictions would not change and performance would not degrade. **It was observed that performance degrades as k increases**, for RoBERTa, BART, GPT-2 and OPT alike.

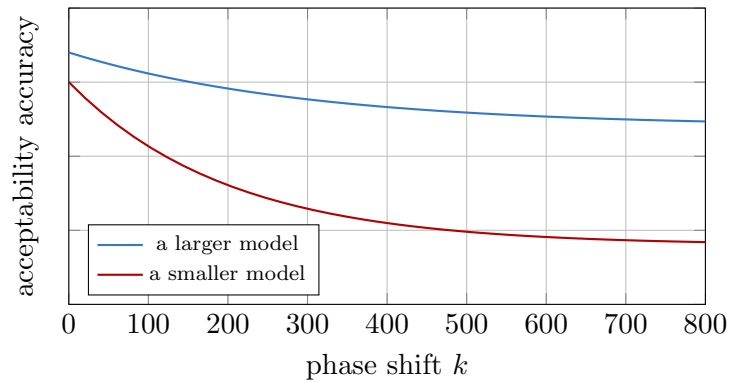


Figure 6: Schematic of the slide’s figure (shapes only): accuracy falls as the start position is shifted further, for every model family.

Key idea

Learned APE does **not** reliably learn relative-position behaviour. This motivates encoding relative positions **explicitly**.

2.3 Limitation 2: no extrapolation beyond the context length

APE cannot extrapolate to sequences longer than the context length T .

Worked example: story generation

A GPT model (APE) was pre-trained with $T = 512$. We want it to generate a 1024-token story. There are **no learned embeddings for positions 513 to 1024**: rows 513..1024 of P simply do not exist. We would have to re-train the model with $T = 1024$, which is computationally expensive. So enabling a model to extrapolate beyond T is important.

2.4 Limitation 3: length generalization (within the context)

Closely related is **length generalization**: extrapolating from *shorter training sequences* to *longer test sequences*, even when both fit inside the context window.

Worked example: learning to add

An LLM with context length 128 is trained on examples that only ever show how to add two **4-digit** integers:

$$\begin{array}{r} 9999 \\ + 0001 \\ \hline 10000 \end{array}$$

Can it generalise the *algorithm* to add two **12-digit** integers? The whole problem still fits in 128 tokens, so the context length is not the obstacle. The type of PE turns out to be a major factor, and APE is **not** a good choice.

Intuition: why APE hurts algorithmic generalization

Addition is a *local, position-relative* rule: “add the digit under me, carry from the digit to my right”. With APE the model has to learn this rule separately for each absolute position it

saw in training (digit positions 1–4). Digits 5–12 sit at positions whose embeddings were never trained in this role, so the rule does not transfer. A relative scheme expresses “the digit k places to my right” in the same way everywhere.

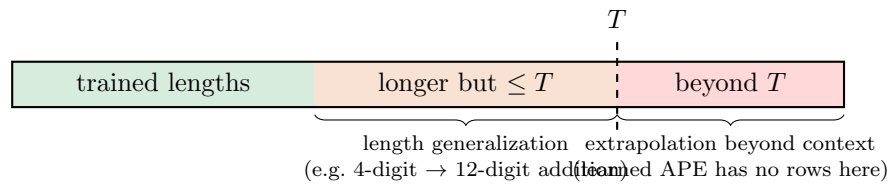


Figure 7: Two related but distinct failure modes of APE.

Summary: Module 1

- Learned APE is used by BERT, RoBERTa, GPT-x, BART and OPT.
- Shifting positions by k degrades performance, so APE does not truly learn relative position.
- APE cannot extrapolate beyond the training context length T (no embeddings exist there).
- APE also generalises poorly to longer inputs within the context (e.g. 4-digit $\rightarrow$ 12-digit addition).

3 Module 2: Relative position encoding methods

3.1 One absolute position vs T relative positions

With APE each token has exactly **one** absolute position. With **Relative Position Encoding (RPE)**, each token has T **relative positions** in a sequence of length T : one with respect to every token, itself included. So RPE encodes **pairwise relationships** between tokens, which APE does not.

	somewhere	something	incredible	is	waiting	to	be	known
APE	①	②	③	④	⑤	⑥	⑦	⑧
RPE w.r.t. "is"	-3	-2	-1	0	1	2	3	4

Figure 8: The relative position of "is" is 0 with respect to itself and differs for every surrounding word: $(-3, -2, -1, 0, 1, 2, 3, 4)$.

Displaying all relative positions $j - i$ for the sequence (row i = query word, column j = key word):

	somewhere	something	incredible	is	waiting	to	be	known
somewhere	0	1	2	3	4	5	6	7
something	-1	0	1	2	3	4	5	6
incredible	-2	-1	0	1	2	3	4	5
is	-3	-2	-1	0	1	2	3	4
waiting	-4	-3	-2	-1	0	1	2	3
to	-5	-4	-3	-2	-1	0	1	2
be	-6	-5	-4	-3	-2	-1	0	1
known	-7	-6	-5	-4	-3	-2	-1	0

Figure 9: The relative-position matrix $j - i$. It is a *Toeplitz* matrix: constant along every diagonal. Blue = key to the right (positive), orange = key to the left (negative).

- Each token has T relative positions, and overall there are $2T - 1$ distinct values, $-(T - 1), \dots, 0, \dots, T - 1$. So RPE needs $2T - 1$ **embeddings**, whereas APE needs only T .
- The pairwise relations form a **labelled, directed, fully-connected graph**. The edge somewhere $\rightarrow$ something has label $+1$; the edge something $\rightarrow$ somewhere has label -1 .

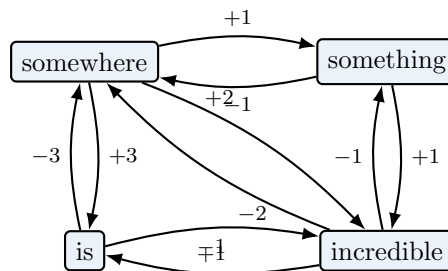


Figure 10: Part of the labelled directed graph for the first four words. Edge labels are relative distances (target position - source position).

3.2 The naive implementation is expensive

Keep a table of $2T-1$ relative embeddings $p_{-(T-1)}, \dots, p_{T-1}$. A naive way to use them is to combine (say, add) all T relative embeddings into each token embedding:

$$h_i = x_i + \sum_{j=0}^{T-1} p_{j-i}, \quad \text{e.g. for } T = 8: \quad h_3 = x_3 + (p_{-3} + p_{-2} + \dots + p_4).$$



Figure 11: Naive RPE: token 3 needs the 8 rows $p_{-3}..p_4$ (highlighted).

For $T = 1024$ that is 1024 additions *per token*. In general it needs T^2 additions instead of the T additions of APE. **Can we do better?**

Common pitfall

Notice also that $\sum_j p_{j-i}$ collapses all pairwise information into one vector per token. Token i no longer knows *which* neighbour is at which distance. The real fix is to use relative positions *inside the pairwise score* e_{ij} , where each pair (i, j) gets its own distance.

3.3 The idea: decompose the pre-attention score

The pre-attention score is $e_{ij} = q_i k_j^\top$. With APE, $q_i = h_i W_Q = (x_i + p_i) W_Q$ and $k_j = h_j W_K = (x_j + p_j) W_K$, so

Derivation: four-term decomposition

$$\begin{aligned} e_{ij} &= (x_i + p_i) W_Q W_K^\top (x_j + p_j)^\top \\ &= (x_i W_Q + p_i W_Q) (W_K^\top x_j^\top + W_K^\top p_j^\top) \\ &= \underbrace{x_i W_Q W_K^\top x_j^\top}_{\text{(a) word-word}} + \underbrace{x_i W_Q W_K^\top p_j^\top}_{\text{(b) word-position}} + \underbrace{p_i W_Q W_K^\top x_j^\top}_{\text{(c) position-word}} + \underbrace{p_i W_Q W_K^\top p_j^\top}_{\text{(d) position-position}}. \end{aligned}$$

Dimensions: $x_i, p_i \in \mathbb{R}^{1 \times d_{\text{model}}}$, $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d}$, so each term is a scalar.

- (a) is the score for the query and key **without any position encoding**.
- (b), (c) are **correlations between a word and a position**.
- (d) is a **constant that comes purely from positions**.

So PE effectively **adds terms to the pre-attention score**. Then we can inject position information *directly* in the attention layer by adjusting e_{ij} , instead of adding vectors at the input.

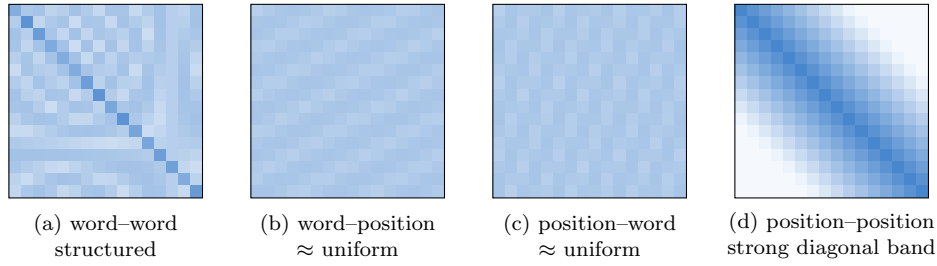


Figure 12: Schematic of the BERT visualisation shown in the slides (from the TUPE paper). The two mixed terms are almost uniformly distributed: word and position show *no correlation*. Adding APE to the input introduces a **positional artefact**, two noisy terms that carry no useful signal.

Key idea

Adding positions to embeddings mixes word and position information in ways that are mostly noise (terms (b), (c)). It is cleaner to leave the content term alone and add a *position-only bias* to e_{ij} that depends on $j - i$.

3.4 Shaw et al. (2018): relative position embeddings in attention

There are many ways to modify the attention score. The simplest is to **add a relative-position vector to the key**:

$$e_{ij} = x_i W_Q (x_j W_K + p_{ij}^K)^\top, \quad p_{ij}^K \in \mathbb{R}^d$$

p_{ij}^K is also called a **position bias** (like the b in $xW + b$). Note that p_{ij} has dimension d (the head dimension), whereas APE vectors have dimension d_{model} .

Does this generalise to unseen lengths? Not yet. If there is a separate p^K for every distance up to $T - 1$, the embedding matrix size ($2T - 1$ rows) still depends on T .

3.4.1 Clipping

Clip the relative distance beyond a maximum distance k :

$$p_{ij}^K = w_{\text{clip}(j-i, k)}^K, \quad \text{clip}(j - i, k) = \max(-k, \min(j - i, k))$$

Now only $2k + 1$ embeddings $w_{-k}^K, \dots, w_k^K$ are needed, **independent of T** . Empirically, clipping does not hurt performance.

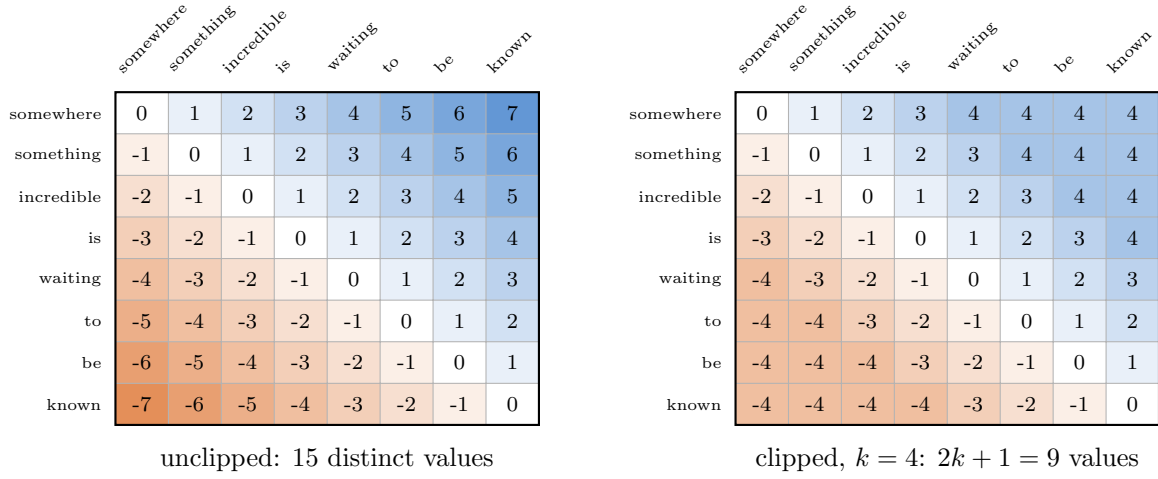


Figure 13: Relative positions for $T = 8$ before and after clipping with $k = 4$. Everything further than 4 tokens away shares the embedding $w_{\pm 4}$. Row “somewhere” becomes $(0, 1, 2, 3, 4, 4, 4, 4)$.

Intuition: why clipping is reasonable

Knowing that a word is “exactly 57 vs 58 tokens away” is rarely useful, while “1 vs 2 tokens away” matters a lot. Clipping keeps fine resolution nearby and lumps all far positions into “far left” / “far right”. Multiple layers still let information travel further than k in total.

3.4.2 The full Shaw attention

$$\begin{aligned}
 e_{ij} &= x_i W_Q (x_j W_K + p_{ij}^K)^\top = x_i W_Q W_K^\top x_j^\top + x_i W_Q (p_{ij}^K)^\top, \\
 \alpha_{ij} &= \text{softmax}_j(e_{ij}) \quad (\text{the paper also divides by } \sqrt{d}), \\
 z_i &= \sum_j \alpha_{ij} (x_j W_V + p_{ij}^V).
 \end{aligned}$$

Relative information is added to the **values** too, through p_{ij}^V (also clipped). The embeddings p_{ij}^K, p_{ij}^V are **shared across heads** but **not across layers**.

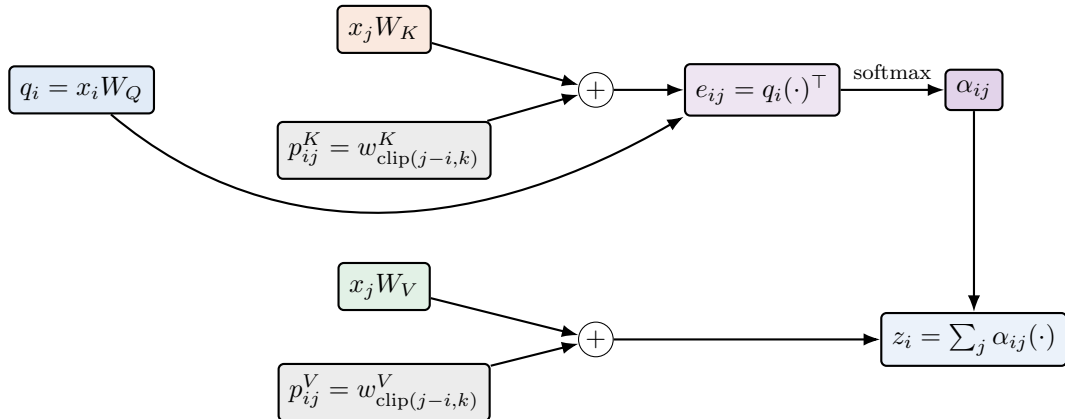


Figure 14: Shaw et al. RPE: learned, clipped relative vectors are added to keys (to change the scores) and to values (to change the output).

Results. Replacing APE in the vanilla Transformer with RPE ($k = 16$) improves machine translation:

Model	Position information	EN-DE BLEU	EN-FR BLEU
Transformer (base)	Absolute	26.5	38.2
Transformer (base)	Relative	26.8	38.7
Transformer (big)	Absolute	27.9	41.2
Transformer (big)	Relative	29.2	41.5

Derivation: efficient computation and parameter count

Only $2k+1$ distinct vectors exist, so $x_i W_Q (p_{ij}^K)^\top$ can be computed as $q_i W_{\text{rel}}^{K\top}$ against the small $(2k+1) \times d$ table, then gathered by distance. Parameters per layer: $2(2k+1)d$ (keys and values). For $k = 16$, $d = 64$: $2 \cdot 33 \cdot 64 = 4224$ parameters, compared with $Td_{\text{model}} = 512 \cdot 512 = 262,144$ for a learned APE table.

3.5 RPE variations

Recall the decomposition $e_{ij} = x_i W_Q W_K^\top x_j^\top + x_i W_Q W_K^\top p_j^\top + p_i W_Q W_K^\top x_j^\top + p_i W_Q W_K^\top p_j^\top$. Different methods replace terms (b), (c), (d) differently.

3.5.1 Transformer-XL (recursively extends context)

$$e_{ij} = \underbrace{x_i W_Q W_K^\top x_j^\top}_{\text{content}} + \underbrace{x_i W_Q R_{j-i}^\top}_{\text{content-position}} + \underbrace{u W_K^\top x_j^\top}_{\text{global content bias}} + \underbrace{v W_K^\top R_{j-i}^\top}_{\text{global position bias}}$$

- R_{j-i} encodes the relative distance between positions i and j , computed from the **sinusoidal** function (non-learnable).
- u, v are **learnable** vectors. They replace $p_i W_Q$ in terms (c) and (d): since only relative position should matter, the query-side absolute position is replaced by a position-independent vector.

Note

In the original paper the position term uses its own projection $W_{K,R}$ (so $R_{j-i} W_{K,R}$ rather than R_{j-i} directly). Transformer-XL needed relative positions because it caches the previous segment's hidden states and reuses them as extra context ("segment-level recurrence"). Absolute positions would clash between segments: position 0 of the current segment and position 0 of the cached one would look identical.

3.5.2 T5 bias

All that is required is adding a constant, so why not just do that?

$$e_{ij} = x_i W_Q W_K^\top x_j^\top + r_{j-i}, \quad r_{j-i} \in \mathbb{R} \text{ learnable scalars.}$$

- The scalars are **shared across layers**. The maximum relative distance is clipped to 128.
- It greatly reduces the number of learnable parameters when the model is scaled: a bias is one number per distance (per head), not a d -dimensional vector.

Note: bucketing in T5

In T5 the distances are mapped to 32 buckets: exact buckets for small distances and logarithmically wider buckets up to 128, beyond which all distances share the last bucket. This is a smarter version of Shaw's clipping. Each head has its own scalar per bucket.

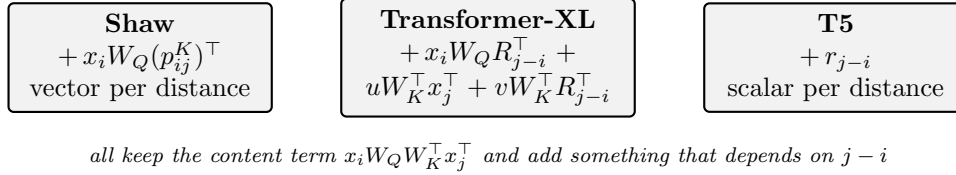


Figure 15: Three relative schemes, from most to least expressive position term.

3.6 Rotary Position Embedding (RoPE)

3.6.1 A unifying view

Line up the methods so far:

$$\begin{aligned} \text{Shaw: } e_{ij} &= \boxed{x_i W_Q W_K^\top x_j^\top} + x_i W_Q (p_{ij}^K)^\top \\ \text{T-XL: } e_{ij} &= \boxed{x_i W_Q W_K^\top x_j^\top} + x_i W_Q R_{j-i}^\top + u W_K^\top x_j^\top + v W_K^\top R_{j-i}^\top \\ \text{T5: } e_{ij} &= \boxed{x_i W_Q W_K^\top x_j^\top} + r_{j-i} \end{aligned}$$

All of them keep the content term and bolt on an *additive* position term. Generalise by writing the query and key as functions of both the token and its position:

$$f_q(x_i, i) = x_i W_Q, \quad f_k(x_j, j) = x_j W_K \quad \Rightarrow \quad \langle f_q(x_i, i), f_k(x_j, j) \rangle = x_i W_Q W_K^\top x_j^\top.$$

(Here the functions ignore position.) What we **want** is for the inner product itself to depend on the positions *only through their difference*:

$$\langle f_q(x_i, i), f_k(x_j, j) \rangle = g(x_i, x_j, j - i)$$

3.6.2 The problem with adding: APE changes angles

Recall $q \cdot k = |q| |k| \cos \theta$. The attention score depends on the *angle* between query and key. Consider “the book” appearing in different places:

Sentence	m (“the”)	n (“book”)
<i>the book ...</i>	0	1
<i>read the book ...</i>	1	2
<i>you read the book ...</i>	2	3
<i>you must read the book ...</i>	3	4
<i>he says you must read the book ...</i>	5	6

The relative distance is always 1, so ideally the relationship between “the” and “book” should look the same every time. With APE ($d = 2$, $p_i = (\sin i, \cos i)$), as in the slide: $p_0 = (0, 1)$, $p_1 = (0.84, 0.54)$, $p_2 = (0.91, -0.42)$, $p_3 = (0.14, -0.99)$, take illustrative embeddings the = $(0.3, 0.2)$, book = $(0.5, -0.1)$:

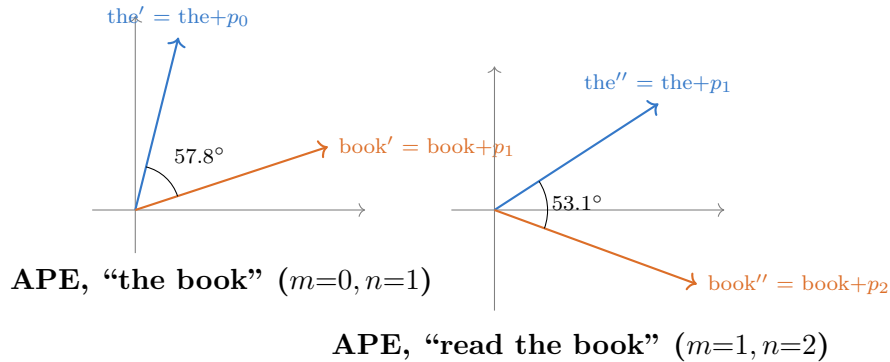


Figure 16: Adding APE changes the angle between “the” and “book” (57.8° vs 53.1° ; also 49.4° , 48.0° , 68.1° for the other sentences) although their relative distance is the same.

Can we preserve the angle between word representations so that relative information is encoded in angles? Yes: *rotate* each vector by an angle proportional to its position. Rotating both by the same extra amount leaves the angle between them unchanged.

3.6.3 Derivation with complex numbers

Assume the following functional form, with rotation in mind (use m, n for positions and j for $\sqrt{-1}$):

$$f_q(x_m, m) = x_m W_Q e^{jm\theta}, \quad f_k(x_n, n) = x_n W_K e^{jn\theta}.$$

We multiply the affine-transformed word embedding by a (complex) constant. For $x, y \in \mathbb{C}^d$ the inner product is $x \cdot y^H$, where H is the Hermitian (conjugate) transpose. Then

Derivation: the inner product depends only on $m - n$

$$\begin{aligned} \langle f_q(x_m, m), f_k(x_n, n) \rangle &= x_m W_Q e^{jm\theta} (x_n W_K e^{jn\theta})^H \\ &= x_m W_Q (x_n W_K)^H e^{jm\theta} \overline{e^{jn\theta}} \quad (\overline{e^{jn\theta}} = e^{-jn\theta}) \\ &= x_m W_Q (x_n W_K)^H e^{j(m-n)\theta}, \\ g(x_m, x_n, m - n) &= \text{Re}[(x_m W_Q)(x_n W_K)^H e^{j(m-n)\theta}]. \end{aligned}$$

The positions enter only as $m - n$. ✓

Note: why θ must be non-zero

If $\theta = 0$ then $e^{jm\theta} = 1$ for every m : no rotation, no position information at all, and we are back to the position-free score. Also, θ should not make $m\theta$ wrap around too quickly, since positions m and $m + 2\pi/\theta$ would look identical. Using many different θ 's (below) removes this ambiguity.

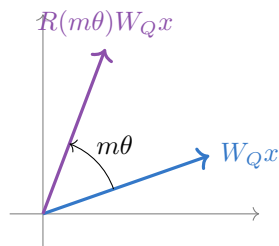
3.6.4 The same thing as a rotation matrix

Using $e^{jm\theta} = \cos(m\theta) + j \sin(m\theta)$, multiplying a 2-D vector (viewed as a complex number) by $e^{jm\theta}$ is the matrix

$$e^{jm\theta} \equiv R(m\theta) = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix},$$

which is the **rotation matrix**. It rotates the affine-transformed x_m by the angle $m\theta$. For $x_m \in \mathbb{R}^2$ (column convention):

$$f_q(x_m, m) = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} W_q^{11} & W_q^{12} \\ W_q^{21} & W_q^{22} \end{bmatrix} \begin{bmatrix} x_m^1 \\ x_m^2 \end{bmatrix} = R(m\theta) W_Q x_m, \quad f_k(x_n, n) = R(n\theta) W_K x_n.$$



Rotation keeps the length:
 $\|R(m\theta)v\| = \|v\|$.
 Only the direction encodes position.

Figure 17: RoPE rotates the projected query (and key) by an angle proportional to its position.

Derivation: the inner product of rotated vectors

Rotation matrices satisfy $R(a)^\top = R(-a)$ and $R(a)R(b) = R(a+b)$. Hence

$$\begin{aligned}
 \langle f_q(x_m, m), f_k(x_n, n) \rangle &= (R(m\theta)W_Q x_m)^\top (R(n\theta)W_K x_n) \\
 &= x_m^\top W_Q^\top R(m\theta)^\top R(n\theta) W_K x_n \\
 &= x_m^\top W_Q^\top \begin{bmatrix} \cos m\theta & \sin m\theta \\ -\sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} \cos n\theta & -\sin n\theta \\ \sin n\theta & \cos n\theta \end{bmatrix} W_K x_n \\
 &= x_m^\top W_Q^\top \begin{bmatrix} \cos((m-n)\theta) & \sin((m-n)\theta) \\ -\sin((m-n)\theta) & \cos((m-n)\theta) \end{bmatrix} W_K x_n \\
 &= x_m^\top W_Q^\top R((n-m)\theta) W_K x_n.
 \end{aligned}$$

Multiplying out the middle product uses $\cos m\theta \cos n\theta + \sin m\theta \sin n\theta = \cos((m-n)\theta)$ and $\cos m\theta \sin n\theta - \sin m\theta \cos n\theta = \sin((n-m)\theta)$. The score depends on the positions only through $n-m$.

Worked example: the angle is preserved

Same toy vectors, with $W_Q = W_K = I$ and $\theta = 1$ rad. Rotate “the” by $m\theta$ and “book” by $n\theta$:

(m, n)	rotated the	rotated book	angle / dot product
(0, 1)	(0.300, 0.200)	(0.354, 0.367)	12.3° / 0.1796
(1, 2)	(-0.006, 0.361)	(-0.117, 0.496)	12.3° / 0.1796
(2, 3)	(-0.307, 0.190)	(-0.481, 0.170)	12.3° / 0.1796
(5, 6)	(0.277, -0.231)	(0.452, -0.236)	12.3° / 0.1796

Every pair has exactly the same angle and dot product, whereas APE gave a different angle for each sentence.

3.6.5 Extending to $d > 2$ dimensions

Split the d -dimensional vector into $d/2$ pairs and rotate each pair with its own frequency θ_i :

$$\begin{aligned}
 f_q(x_m, m) &= R_{\Theta, m}^d W_Q x_m \\
 R_{\Theta, m}^d &= \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{bmatrix} \\
 \Theta &= \left\{ \theta_i = 10000^{-2(i-1)/d}, i = 1, 2, \dots, d/2 \right\}
 \end{aligned}$$

These are the same frequencies as in sinusoidal embeddings. Then the pre-attention score encodes the relative position:

$$\begin{aligned}
 q_m^\top k_n &= (R_{\Theta, m}^d W_Q x_m)^\top (R_{\Theta, n}^d W_K x_n) && \leftarrow \text{APE of } m, n \\
 &= x_m^\top W_Q^\top (R_{\Theta, m}^d)^\top R_{\Theta, n}^d W_K x_n \\
 &= x_m^\top W_Q^\top R_{\Theta, n-m}^d W_K x_n && \leftarrow \text{RPE of the difference}
 \end{aligned}$$

since each 2×2 block satisfies $R(m\theta_i)^\top R(n\theta_i) = R((n-m)\theta_i)$.

Note: sign conventions

The slide writes the final matrix as $R_{\Theta, m-n}^d$ with $R_{\Theta, n-m}^\top = (R_{\Theta, m}^d)^\top R_{\Theta, n}^d$. That is the same statement, since $R(a)^\top = R(-a)$. Whether you call it $m-n$ or $n-m$ is a convention; what matters is that only the difference appears.

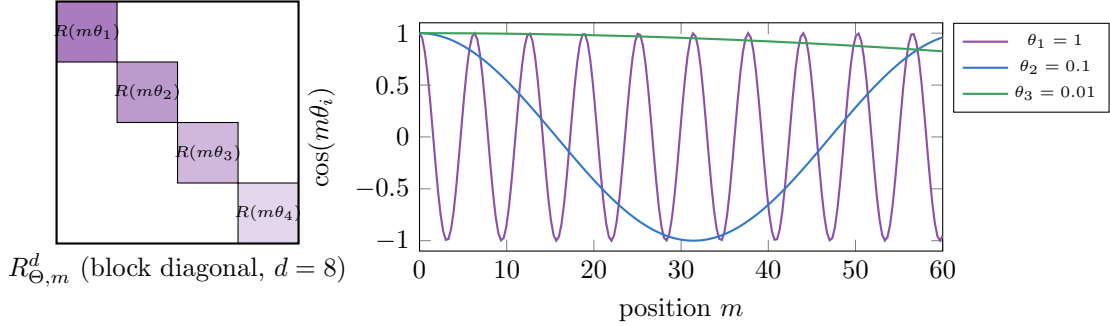


Figure 18: Left: $R_{\Theta, m}^d$ has one 2×2 rotation per pair of dimensions. Right: the pairs rotate at geometrically decreasing speeds ($d=8$: $\theta_i = 10000^{-2(i-1)/8} = 1, 0.1, 0.01, 0.001$). Fast pairs resolve nearby positions, slow pairs resolve distant ones.

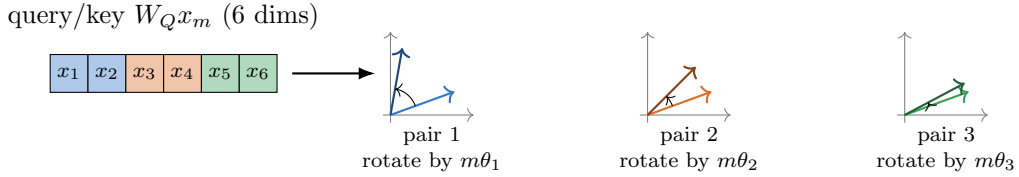


Figure 19: Implementation view of RoPE (as in the RoFormer figure): consecutive dimension pairs of the projected query/key are each rotated by position $\times$ their own frequency.

Derivation: efficient $\mathcal{O}(d)$ implementation

$R_{\Theta, m}^d$ is sparse, so never multiply by the full $d \times d$ matrix. For $v = W_Q x_m = (v_1, \dots, v_d)$:

$$R_{\Theta, m}^d v = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ \vdots \\ v_{d-1} \\ v_d \end{bmatrix} \odot \begin{bmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{bmatrix} + \begin{bmatrix} -v_2 \\ v_1 \\ -v_4 \\ v_3 \\ \vdots \\ -v_d \\ v_{d-1} \end{bmatrix} \odot \begin{bmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{bmatrix}.$$

Check pair 1: $v_1 \cos m\theta_1 - v_2 \sin m\theta_1$ and $v_2 \cos m\theta_1 + v_1 \sin m\theta_1$, exactly $R(m\theta_1)(v_1, v_2)^\top$. The cost is $\mathcal{O}(d)$ per vector, and the cos/sin tables can be precomputed for all positions.

3.6.6 Properties and performance

- **Relative by construction:** $q_m^\top k_n$ depends on $n-m$ only.

- **Norm preserving:** rotations do not change $\|q\|, \|k\|$, so there are no extra noise terms like APE's word-position cross terms.
- **No parameters:** fixed formula, nothing to learn.
- **Long-term decay:** the RoFormer paper shows that an upper bound on $|q_m^\top k_n|$ shrinks as $|m - n|$ grows (averaged over the many frequencies), which matches the intuition that far tokens should matter less.
- **Applied at every layer, to queries and keys only:** unlike sinusoidal APE (added once at the input), RoPE injects position information in **every** attention layer, and the position embedding is **not** injected into the value vectors.
- **Faster pre-training:** in the RoFormer experiments, the MLM loss of RoFormer decreases faster than BERT's, and a Performer with RoPE reaches lower LM loss than without it.

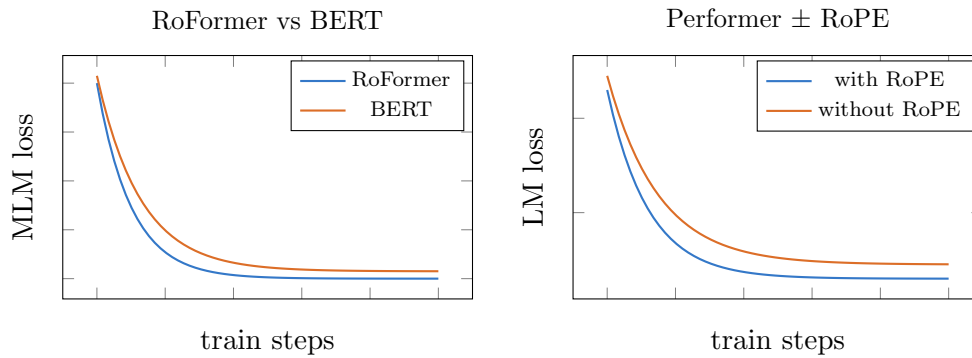


Figure 20: Schematic of the slide's pre-training curves (shapes only).

Summary: Module 2

- RPE encodes pairwise distances $j-i$; there are $2T-1$ of them, and a naive implementation costs T^2 additions.
- Expanding e_{ij} shows APE only adds terms to the score, two of which are noise. So add position terms to e_{ij} directly.
- Shaw: $e_{ij} = x_i W_Q (x_j W_K + p_{ij}^K)^\top$, plus p_{ij}^V on values; clip at k , giving $2k+1$ vectors independent of T .
- Transformer-XL: sinusoidal R_{j-i} plus learnable u, v . T5: a learnable scalar bias r_{j-i} per (bucketed) distance.
- RoPE: rotate q and k by position-dependent angles; $q_m^\top k_n = x_m^\top W_Q^\top R_{\Theta, n-m}^d W_K x_n$, relative, multiplicative, parameter-free, applied at every layer.

4 Module 3: Position encoding and length generalization

4.1 The question

So far we have seen PE methods that lead to better performance. **How do they help in length generalization?** Define length generalization as *the ability of a model to extrapolate to sequences longer than those it saw during training*.

The original Transformer paper chose sinusoidal PE over learned PE (the two gave nearly identical results) “because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.” However, **no experiments were conducted** to validate this. So which approach, APE or RPE, is actually good at extrapolation?

4.2 The extrapolation experiment (Press et al., ALiBi paper)

- Train a **causal language model** with $T = 512$ tokens.
- At inference, let the model predict next tokens up to T_{valid} tokens, and measure quality with **perplexity**.
- Vary T_{valid} from 512 to 16,000 tokens.
- Repeat the experiment changing **only** the position encoding (sinusoidal, T5 bias, RoPE, ALiBi), keeping everything else fixed.

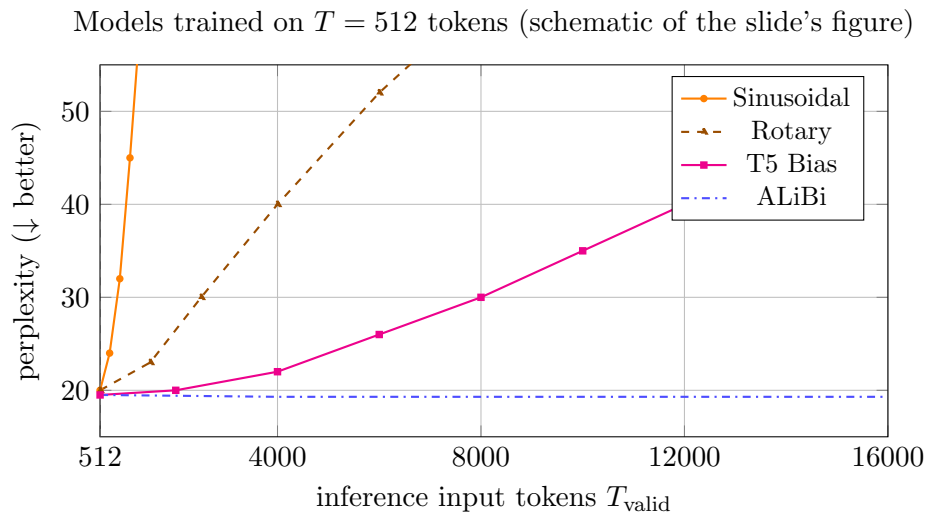


Figure 21: Qualitative picture: sinusoidal APE collapses almost immediately beyond 512 tokens, rotary somewhat later, T5 bias degrades gradually, and ALiBi stays flat even at 16K tokens.

Does increasing the training context to $T = 1024$ help? Yes, and it helps the **T5-bias RPE** more than rotary and sinusoidal APE. For example, at $T_{\text{valid}} = 8000$ the model trained with $T = 512$ has a higher perplexity than the one trained with $T = 1024$. This suggests that **modifying T5 bias** might improve extrapolation. That is ALiBi.

4.3 Attention with Linear Biases (ALiBi)

Recall the T5-bias RPE: $e_{ij} = x_i W_Q W_K^\top x_j^\top + r_{j-i}$ with learnable scalars r_{j-i} .

- **Why not hard-code r_{j-i} ?** A fixed formula means no learned bias table and no bucketing lookup, which also helps inference speed.
- **How?** Just add a constant **negative** bias to the attention score, proportional to the distance.

For a causal model the score of query i is

$$\text{softmax} \left(q_i K^\top + m \cdot (- (i-1), \dots, -2, -1, 0) \right)$$

i.e. the bias for key $j \leq i$ is $-m(i-j)$.

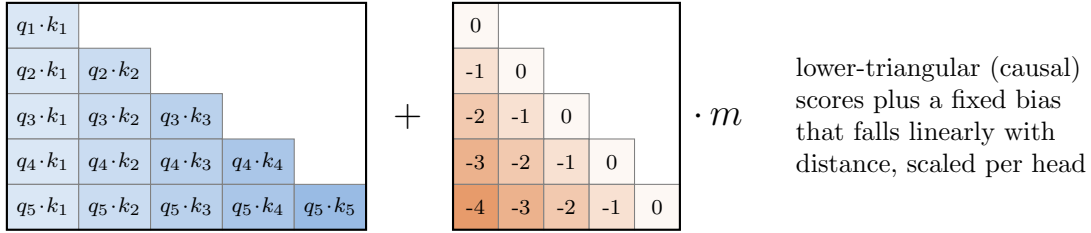


Figure 22: ALiBi: the bias matrix has 0 on the diagonal, -1 on the first sub-diagonal, -2 on the next, and so on, multiplied by the head’s slope m .

- m is a **head-specific scalar** that follows a **geometric progression**. For $n = 8$ heads:
 $m_i = \frac{1}{2^i}$, $i = 1, \dots, n$, i.e. $\frac{1}{2}, \frac{1}{4}, \dots, \frac{1}{256}$.
- Since it is an RPE, position information is added at **every layer**. There is no position embedding at the input.
- It is called **linear** because the bias grows linearly with distance.

Note: slopes for other head counts

The general rule in the ALiBi paper is a geometric sequence starting at $2^{-8/n}$ with ratio $2^{-8/n}$:
 $m_i = 2^{-8i/n}$. For $n = 8$ this gives $1/2^i$. For $n = 16$: $2^{-0.5}, 2^{-1}, 2^{-1.5}, \dots, 2^{-8}$. The slopes are not learned (learning them did not help extrapolation).

4.3.1 ALiBi acts like local windowed attention

Why does it work so well, even at $T_{\text{valid}} = 16\text{K}$? The bias for nearby tokens $(0, -1, -2, \dots)$ is much larger than for distant ones $(-128, -129, \dots)$. After the softmax, $e^{-m(i-j)}$ multiplies the weight of a key at distance $i-j$, so distant keys are exponentially suppressed. This acts as a **soft local window**: the model predicts the next token mainly from the recent past k tokens, which is a natural structure in language. Sequences longer than those seen in training therefore look “familiar”: the model never relied on far-away positions anyway.

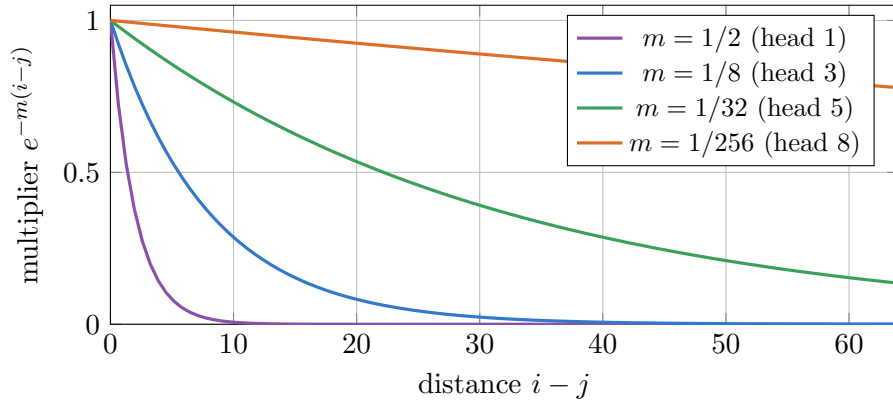


Figure 23: How strongly ALiBi down-weights a key at a given distance, per head (before content scores). Steep heads are very local; shallow heads look far back. The geometric slopes give a spread of window sizes.

Worked example: effective window

A key is effectively ignored once its bias is about -10 (a factor $e^{-10} \approx 4.5 \times 10^{-5}$) unless its content score is unusually high. The distance where $m(i-j) = 10$ is $10/m$: 20 tokens for $m = 1/2$, 80 for $m = 1/8$, 320 for $m = 1/32$, 2560 for $m = 1/256$. So an 8-head ALiBi layer contains heads with windows from about 20 to about 2500 tokens.

4.3.2 Beyond ALiBi

Subsequent studies generalised ALiBi, e.g. **KERPLE** (Kernelized Relative Position Embedding for Length Extrapolation), and improved RoPE, e.g. **xPOS**, which adds an exponential decay to rotary embeddings.

4.4 Is that all? No position encoding (NoPE)

A study (*NoPos*) conjectures that the **causal mask implicitly encodes position information**: a decoder-only model can work without *any* explicit PE.

$q_1 \cdot k_1$				
$q_2 \cdot k_1$	$q_2 \cdot k_2$			
$q_3 \cdot k_1$	$q_3 \cdot k_2$	$q_3 \cdot k_3$		
$q_4 \cdot k_1$	$q_4 \cdot k_2$	$q_4 \cdot k_3$	$q_4 \cdot k_4$	
$q_5 \cdot k_1$	$q_5 \cdot k_2$	$q_5 \cdot k_3$	$q_5 \cdot k_4$	$q_5 \cdot k_5$

row i has exactly i visible keys:
the *shape* of the mask differs per position,
so position can be inferred

Figure 24: The causal mask: token i can only see tokens $1..i$.

Derivation: how a causal model can count

Suppose some head gives equal scores to all visible keys (e.g. $W_Q = 0$), and the values carry a constant feature, e.g. the value of the **[bos]** token is 1 and all others are 0. Then at position i

$$z_i = \sum_{j=1}^i \frac{1}{i} v_j = \frac{1}{i}.$$

The output encodes the absolute position i (as $1/i$). Later layers can turn this into any positional feature they need. Without the causal mask (bidirectional attention) every token would see the same set and get the same average, so encoders do need explicit PE.

Evidence 1: perplexity.

Validation perplexity	WikiText-103	The Pile
NoPos	20.97	13.10
Learned	20.42	13.05
Sinusoidal	20.16	12.93
ALiBi	19.71	12.51

NoPE is only slightly worse than models with explicit PE.

Evidence 2: downstream length generalization. ALiBi used **perplexity** to measure length generalization. That may not suit downstream tasks like **sorting, addition, summation, reversing**, where one wrong token ruins the answer. Keeping both observations in mind, one study (Kazemnejad et al., 2023) extended the length-generalization analysis of decoder-only models to such tasks, comparing sinusoidal APE, RoPE, T5 bias, ALiBi and NoPE:

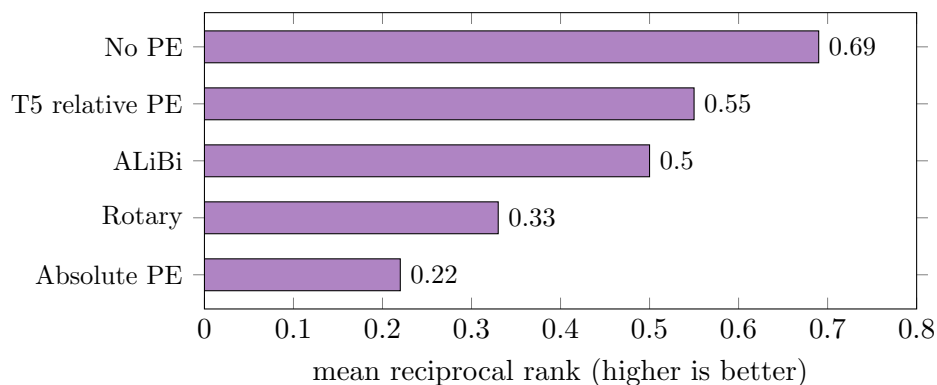


Figure 25: Values from the slide’s figure. NoPE ranks best at length generalization across the tasks, and absolute PE worst. The encodings used in GPT-3-style LLMs (ALiBi in BLOOM, RoPE in PaLM and LLaMA) sit in the middle.

Theory. It is shown theoretically that NoPE (i.e. decoder-only models without PE)

1. **can represent both absolute and relative PEs**, and
2. **learns to use relative PE in practice.**

4.5 Is there an inherent limitation?

Does the Transformer’s design itself prevent effective length generalization? A recent study (Feb 2024, Google DeepMind: *Transformers Can Achieve Length Generalization But Not Robustly*) suggests so: with the right PE and data format, a Transformer can extrapolate (e.g. on addition), but the ability is **fragile**, sensitive to random seed and training-data order. Despite all these studies, **length generalization remains a challenge** for Transformer-based models.

4.6 Summary of position encodings

	PE	Manifestation	Learnable	Integration	Injection layer
APE	Sinusoidal (Vaswani et al., 2017)	Embedding	no	Add	Initial
	SHAPE (Kiyono et al., 2021)	Embedding	no	Add	Initial
	CAPE (Likhomanenko et al., 2021)	Embedding	no	Add	Initial
	Complex (Wang et al., 2021)	Embedding	yes	Multiply	Initial
	FLOATER (Liu et al., 2020)	Embedding	yes	Add	Initial
RPE	Shaw et al. (2018)	Embedding	yes	Add	Every
	T5 Bias (Raffel et al., 2020)	Bias	yes	Add	Every
	ALiBi (Press et al., 2022)	Bias	no	Add	Every
	KERPLE (Chi et al., 2022)	Bias	yes	Add	Every
	SANDWICH (Chi et al., 2023)	Embedding	no	Add	Every
	FIRE (Li et al., 2023)	Bias	yes	Add	Every
	Transformer-XL (Dai et al., 2019)	Embedding	no	Add	Every
	RoPE (Su et al., 2024)	Embedding	no	Multiply	Every
	xPOS (Sun et al., 2023)	Embedding	no	Multiply	Every

NoPE is not in the list for obvious reasons (it could be listed under RPE, since it ends up using relative positions).

Note: reading the table

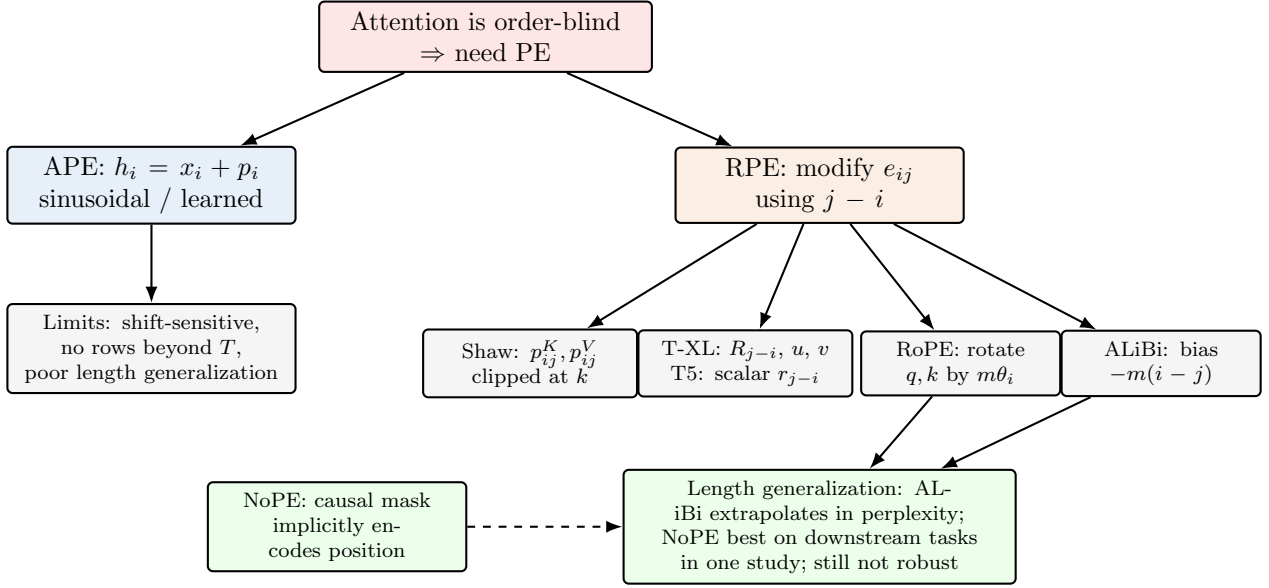
Manifestation: whether position shows up as a vector (embedding) or as a scalar added to scores (bias). **Integration:** added or multiplied (rotations). **Injection layer:** every APE injects once at the input; every RPE injects at every layer. That last column is the clearest dividing line between the two families. (The table marks Transformer-XL as not learnable because its R_{j-i} is sinusoidal, even though u, v are learned.)

Summary: Module 3

- The Transformer paper hoped sinusoidal PE would extrapolate but never tested it.
- Trained at 512 and tested up to 16K: sinusoidal fails fast, rotary later, T5 bias degrades gradually, ALiBi stays flat.
- ALiBi: fixed bias $-m(i - j)$, geometric per-head slopes ($1/2^i$ for 8 heads), every layer, no input PE. It works like a soft local window.
- NoPE: the causal mask implicitly encodes position. It has close perplexity and the best downstream length generalization in one study, and it can represent both APE and RPE, using RPE in practice.
- Length generalization is achievable but not robust, and remains an open problem.

5 Consolidated summary and formula sheet

5.1 Concept map



5.2 Formula sheet

Item	Formula
Sinusoidal APE	$PE(i, 2k) = \sin(i\omega_k)$, $PE(i, 2k + 1) = \cos(i\omega_k)$, $\omega_k = 10000^{-2k/d_{\text{model}}}$; $h_i = x_i + p_i$
Shift property	$(\sin, \cos)$ at $i + k = \text{rotation by } k\omega \text{ of } (\sin, \cos) \text{ at } i$
Relative positions	$j - i \in \{-(T - 1), \dots, T - 1\}$: $2T - 1$ values; naive RPE costs T^2 additions
Score decomposition	$e_{ij} = x_i W_Q W_K^\top x_j^\top + x_i W_Q W_K^\top p_j^\top + p_i W_Q W_K^\top x_j^\top + p_i W_Q W_K^\top p_j^\top$
Shaw	$e_{ij} = x_i W_Q (x_j W_K + p_{ij}^K)^\top$, $z_i = \sum_j \alpha_{ij} (x_j W_V + p_{ij}^V)$, $p_{ij} = w_{\text{clip}(j-i, k)}$, $2k + 1$ vectors
Clip	$\text{clip}(j - i, k) = \max(-k, \min(j - i, k))$
Transformer-XL	$e_{ij} = x_i W_Q W_K^\top x_j^\top + x_i W_Q R_{j-i}^\top + u W_K^\top x_j^\top + v W_K^\top R_{j-i}^\top$
T5 bias	$e_{ij} = x_i W_Q W_K^\top x_j^\top + r_{j-i}$, scalar, shared across layers, max distance 128
RoPE (2-D)	$f_q = R(m\theta) W_Q x_m$, $f_k = R(n\theta) W_K x_n$, $q_m^\top k_n = x_m^\top W_Q^\top R((n - m)\theta) W_K x_n$
RoPE (d -D)	$R_{\Theta, m}^d = \text{blockdiag}(R(m\theta_1), \dots, R(m\theta_{d/2}))$, $\theta_i = 10000^{-2(i-1)/d}$
ALiBi	$\text{softmax}(q_i K^\top + m \cdot (-(i - 1), \dots, -1, 0))$, $m_i = 2^{-8i/n}$ ($= 1/2^i$ for $n = 8$)
NoPE counting	uniform attention over the causal prefix: $z_i = \frac{1}{i} \sum_{j \leq i} v_j$

5.3 Comparison at a glance

Method	Absolute/relative	Learned?	Where	Params	Beyond T ?
Sinusoidal APE	absolute	no	input, add	0	poor
Learned APE	absolute	yes	input, add	Td_{model}	impossible
Shaw RPE	relative	yes	every layer, + (K, V)	$2(2k+1)d/\text{layer}$	ok (clipped)
T5 bias	relative	yes	every layer, score	buckets $\times n_h$	moderate
RoPE	relative	no	every layer, $\times (Q, K)$	0	degrades
ALiBi	relative	no	every layer, score	0	good (perplexity)
NoPE	(implicit)	no	—	0	best on some tasks

5.4 Ten sentences to remember

1. Attention is permutation-equivariant, so order must be injected.
2. Vanilla APE adds fixed sinusoids once at the input; learned APE uses a $T \times d_{\text{model}}$ table (BERT, GPT, BART, OPT).
3. Shifting positions by k hurts APE models, so they do not truly learn relative position.
4. APE cannot extrapolate beyond T and generalises poorly to longer inputs (e.g. 4-digit $\rightarrow$ 12-digit addition).
5. RPE gives each pair a distance $j - i$; a naive implementation costs T^2 additions, so inject positions into e_{ij} instead.
6. Expanding e_{ij} with APE gives word–position cross terms that are essentially noise (a positional artefact).
7. Shaw adds clipped learned vectors to keys and values; T5 adds a learned scalar; Transformer-XL uses sinusoidal R_{j-i} with learned u, v .
8. RoPE rotates q, k by position so that $q_m^\top k_n$ depends only on $n - m$; it has no parameters and is applied at every layer, not to values.
9. ALiBi adds $-m \cdot \text{distance}$ with geometric head slopes, acts like a soft local window, and extrapolates to 16K when trained on 512.
10. NoPE works in decoders because the causal mask leaks position; length generalization remains an open, fragile problem.

6 Practice set

Questions first, solutions afterwards. Angles are in radians unless marked with $^\circ$.

6.1 Part A: Numerical problems

- N1. Sinusoidal APE.** With $d_{\text{model}} = 4$ (frequencies $\omega_0 = 1$, $\omega_1 = 10000^{-2/4}$), compute the position vector p_2 using the standard pairing $(\sin, \cos, \sin, \cos)$.
- N2. Shift as rotation.** Using only p_2 's first pair from N1 and the rotation property, compute the first pair of p_3 . Verify directly.
- N3. Learned APE parameters.** BERT-base has $T = 512$, $d_{\text{model}} = 768$. How many parameters does its learned APE table have? A GPT with $T = 512$ must support 1024 tokens: how many new rows and parameters are needed, and why can't they just be "filled in"?
- N4. Relative-position matrix.** For $T = 6$, write row $i = 2$ of the matrix $j - i$. How many distinct relative positions exist in total? How many does APE need?
- N5. Naive RPE cost.** How many vector additions does naive RPE ($h_i = x_i + \sum_j p_{j-i}$) need for $T = 1024$, compared with APE?
- N6. Score decomposition.** Take $W_Q W_K^\top = I_2$, $x_i = (1, 0)$, $p_i = (0, 1)$, $x_j = (0.5, 0.5)$, $p_j = (1, -1)$. Compute the four terms of e_{ij} and their sum. Verify against $(x_i + p_i)(x_j + p_j)^\top$.
- N7. Clipping.** $T = 10$, $k = 3$. Write the clipped relative positions for row $i = 0$ and row $i = 6$. How many distinct embeddings are needed?
- N8. Shaw table check.** For $T = 8$, $k = 4$, verify the row for "to" ($i = 5$) in the slide's clipped matrix.
- N9. Parameter counts.** A 12-layer model with $d = 64$ uses Shaw RPE with $k = 16$ for keys and values (shared across heads, not layers). Count the RPE parameters. Compare with T5 bias using 32 buckets, 12 heads, shared across layers.
- N10. RoPE in 2-D.** $\theta = \pi/6$, $W_Q = W_K = I$. Query $q = (1, 0)$ at $m = 2$, key $k = (0, 1)$ at $n = 5$. Compute the rotated vectors and their dot product. Verify with $q^\top R((n - m)\theta)k$.
- N11. RoPE invariance.** Repeat N10 with $m = 7$, $n = 10$. What do you expect and why?
- N12. RoPE frequencies.** For $d = 8$ list $\theta_1.. \theta_4$ and the wavelengths $2\pi/\theta_i$. For $d = 128$ compute θ_{64} and its wavelength.
- N13. ALiBi slopes.** List the slopes for $n = 8$ heads, and the first four for $n = 16$ heads.
- N14. ALiBi bias matrix.** Write the 4×4 causal ALiBi bias matrix for a head with $m = 1/2$.
- N15. ALiBi softmax.** Query 4 has equal content scores $q_4 \cdot k_j = 2$ for $j = 1..4$, head slope $m = 1/2$. Compute the attention weights with and without ALiBi.
- N16. ALiBi with content.** Content scores $(1, 3, 0, 2)$ for keys 1..4 (query 4), $m = 1/2$. Compute the weights with and without the bias. Which key's weight changes most?
- N17. Effective window.** For $m = 1/16$, at what distance is the bias -10 ? What multiplicative factor does that give?

- N18. NoPE counting.** A causal head with uniform attention and values $v_1 = 1$, $v_{j>1} = 0$. What is z_4 ? How far apart are z_{99} and z_{100} , and what does that suggest?
- N19. Complex RoPE.** Treat 2-D vectors as complex numbers: $q = 1 + 0j$ at $m = 1$, $k = 0 + 1j$ at $n = 3$, $\theta = \pi/4$. Compute $\text{Re}[qe^{jm\theta}ke^{jn\theta}]$.
- N20. Extrapolation reading.** In the ALiBi experiment ($T = 512$), by what factor is $T_{\text{valid}} = 16000$ longer than training? If ALiBi's perplexity stays at about 19.3, what does that say compared with sinusoidal PE?

6.2 Part A: Solutions

- N1.** $\omega_0 = 1$, $\omega_1 = 10000^{-0.5} = 0.01$. $p_2 = (\sin 2, \cos 2, \sin 0.02, \cos 0.02) = (0.9093, -0.4161, 0.0200, 0.9998)$.
- N2.** With $k = 1$, $\omega = 1$: $\begin{bmatrix} \sin 3 \\ \cos 3 \end{bmatrix} = \begin{bmatrix} \cos 1 & \sin 1 \\ -\sin 1 & \cos 1 \end{bmatrix} \begin{bmatrix} 0.9093 \\ -0.4161 \end{bmatrix} = \begin{bmatrix} 0.5403 \cdot 0.9093 + 0.8415 \cdot (-0.4161) \\ -0.8415 \cdot 0.9093 + 0.5403 \cdot (-0.4161) \end{bmatrix} = \begin{bmatrix} 0.1411 \\ -0.9900 \end{bmatrix}$. Directly: $\sin 3 = 0.1411$, $\cos 3 = -0.9900$. ✓
- N3.** $512 \times 768 = 393,216$ parameters. Supporting 1024 needs 512 new rows, i.e. another 393,216 parameters. Learned rows are independent parameters with no formula relating them, so the new rows would be untrained and the model would have to be (re-)trained with $T = 1024$.
- N4.** Row $i = 2$: $(-2, -1, 0, 1, 2, 3)$. Distinct values: $-5..5$, that is $2T - 1 = 11$. APE needs $T = 6$.
- N5.** Each token sums T vectors: $T^2 = 1,048,576$ additions vs $T = 1024$ for APE (one addition per token), $1024\times$ more.
- N6.** (a) $x_i \cdot x_j = 0.5$; (b) $x_i \cdot p_j = 1$; (c) $p_i \cdot x_j = 0.5$; (d) $p_i \cdot p_j = -1$. Sum = 1.0. Check: $(1, 1) \cdot (1.5, -0.5) = 1.5 - 0.5 = 1.0$. ✓
- N7.** Row 0: $j - i = 0..9$, clipped: $(0, 1, 2, 3, 3, 3, 3, 3, 3)$. Row 6: $j - i = -6..3$, clipped: $(-3, -3, -3, -3, -2, -1, 0, 1, 2, 3)$. Distinct embeddings: $2k + 1 = 7$.
- N8.** $j - i$ for $i = 5$: $(-5, -4, -3, -2, -1, 0, 1, 2)$; clipping at 4 gives $(-4, -4, -3, -2, -1, 0, 1, 2)$, matching the slide.
- N9.** Shaw: per layer $2 \times (2k + 1) \times d = 2 \times 33 \times 64 = 4224$; $\times 12$ layers = 50,688. T5: $32 \times 12 = 384$ scalars in total (shared across layers), over $100\times$ fewer.
- N10.** $R(2\theta)q$: angle 60° , so $(0.5, 0.866)$. $R(5\theta)k$: k is at 90° , rotated by 150° to 240° , so $(-0.5, -0.866)$. Dot product = $-0.25 - 0.75 = -1$. Check: $R((5 - 2)\theta) = R(\pi/2)$, $R(\pi/2)k = (-1, 0)$, and $q \cdot (-1, 0) = -1$. ✓
- N11.** $n - m = 3$ again, so the dot product is again -1 . RoPE scores depend only on $n - m$. (Explicitly: q rotated to 210° gives $(-0.866, -0.5)$; k rotated from 90° by 300° to $390^\circ \equiv 30^\circ$ gives $(0.866, 0.5)$; dot = -1 .)
- N12.** $d = 8$: $\theta_i = 10000^{-2(i-1)/8}$ gives $1, 0.1, 0.01, 0.001$, with wavelengths 6.28, 62.8, 628, 6283 positions. $d = 128$: $\theta_{64} = 10000^{-126/128} \approx 1.155 \times 10^{-4}$, wavelength $\approx 54,400$ positions.
- N13.** $n = 8$: $1/2, 1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256$. $n = 16$: $2^{-0.5} \approx 0.707$, 0.5 , $2^{-1.5} \approx 0.354$, $0.25, \dots$

N14. $m \cdot \begin{bmatrix} 0 & & & \\ -1 & 0 & & \\ -2 & -1 & 0 & \\ -3 & -2 & -1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & & & \\ -0.5 & 0 & & \\ -1 & -0.5 & 0 & \\ -1.5 & -1 & -0.5 & 0 \end{bmatrix}$ (blank = masked, $-\infty$).

N15. Without ALiBi: uniform, 0.25 each. With ALiBi, biases $(-1.5, -1, -0.5, 0)$: $e^{\text{bias}} = (0.2231, 0.3679, 0.6065, 1)$, sum 2.1975, giving weights $(0.1015, 0.1674, 0.2760, 0.4551)$. Recent keys win.

N16. Without: $\text{softmax}(1, 3, 0, 2) = (0.0871, 0.6439, 0.0321, 0.2369)$. With: $\text{softmax}(-0.5, 2, -0.5, 2) = (0.0379, 0.4621, 0.0379, 0.4621)$. Key 2 (strong content but 2 steps back) drops the most, from 0.644 to 0.462, and key 4 (the most recent) rises from 0.237 to 0.462.

N17. $d = 10/m = 160$ tokens; factor $e^{-10} \approx 4.5 \times 10^{-5}$.

N18. $z_4 = \frac{1}{4}(1 + 0 + 0 + 0) = 0.25$. $z_{99} - z_{100} = \frac{1}{99} - \frac{1}{100} \approx 1.0 \times 10^{-4}$. Positions become harder to tell apart as the sequence grows, one intuition for why implicit positional information gets weaker for long inputs.

N19. $qe^{j\pi/4} \overline{ke^{j3\pi/4}} = q\bar{k} e^{j(1-3)\pi/4} = (1)(-j) e^{-j\pi/2} = (-j)(-j) = -1$, so $\text{Re} = -1$. As a real-vector check: q rotated to 45° , k rotated from 90° to 225° , and $\cos(180^\circ) = -1$. ✓

N20. $16000/512 \approx 31\times$ longer. Flat perplexity means ALiBi extrapolates, whereas sinusoidal APE's perplexity blows up within a few hundred tokens past 512.

6.3 Part B: Subjective questions

- S1.** Why does a Transformer need positional information at all? Prove that self-attention without PE is permutation-equivariant.
- S2.** Describe sinusoidal APE and explain the intuition behind using multiple frequencies. What property did the authors hope would allow extrapolation?
- S3.** Describe the shift experiment on APE models. What was the finding and what did it motivate?
- S4.** Distinguish “extrapolation beyond the context length” from “length generalization”. Give an example of each and explain why APE handles both poorly.
- S5.** How many relative positions does a token have, and how many embeddings does RPE need? Why is the naive implementation expensive, and what is lost by summing?
- S6.** Derive the four-term decomposition of e_{ij} under APE. Interpret each term and explain what the BERT visualisation reveals.
- S7.** Explain Shaw et al.’s relative position representation, including clipping and the value term. Why does clipping make the method independent of T ?
- S8.** Compare Transformer-XL’s and T5’s relative schemes. Why did Transformer-XL need relative positions?
- S9.** Motivate RoPE using the angle between “the” and “book”. What goes wrong with additive APE?
- S10.** Derive, using complex numbers, that RoPE’s inner product depends only on $m - n$. Why must $\theta \neq 0$?
- S11.** Write $R_{\Theta, m}^d$ and show $q_m^\top k_n = x_m^\top W_Q^\top R_{\Theta, n-m}^d W_K x_n$. How is RoPE implemented efficiently?
- S12.** Contrast RoPE and sinusoidal APE with respect to where and how position is injected.
- S13.** Explain ALiBi and why it extrapolates. How are the head slopes chosen?
- S14.** What is NoPE and how can a causal decoder infer positions without PE? Summarise the empirical findings.
- S15.** “Length generalization is solved.” Critically evaluate this claim.

6.4 Part B: Model answers

- S1.** Attention computes weighted averages over a set: scores depend only on token contents. If the input is permuted by P , then Q, K, V become PQ, PK, PV , and $\text{softmax}(PQK^\top P^\top)PV = P \text{softmax}(QK^\top)V$, because a row-wise softmax commutes with permuting rows and columns and $P^\top P = I$. So the outputs are merely permuted, each token’s representation is unchanged, and word order is invisible. Position encodings break this symmetry.
- S2.** p_i has pairs $(\sin i\omega_k, \cos i\omega_k)$ with $\omega_k = 10000^{-2k/d_{\text{model}}}$, added to x_i once at the input. Many geometrically spaced frequencies act like the digits of a clock: fast ones separate neighbours, slow ones separate distant positions, and together they give every position a unique, bounded code. The hoped-for property is that p_{i+k} is a fixed linear map (a rotation depending on k) of p_i , so relative offsets are easy to learn and, in principle, positions beyond training could be handled. The hope was never tested, and later experiments showed sinusoidal APE extrapolates poorly.

- S3.** A pre-trained APE model (e.g. GPT-2) is evaluated on grammatical acceptability via perplexity, but every position index is shifted by k (first token at position k). Relative distances are unchanged, so a model relying on relative position should be unaffected. Instead, performance degrades as k grows, across RoBERTa, BART, GPT-2 and OPT. So learned APE does not truly learn relative position, which motivated encoding relative positions explicitly.
- S4.** Extrapolation beyond context: running at lengths larger than the training context T . Example: a GPT trained with $T = 512$ asked to write 1024 tokens has no learned embeddings for positions 513–1024. Length generalization: training on shorter instances and testing on longer ones, possibly still within T . Example: training on 4-digit addition and testing on 12-digit addition with context 128. APE fails the first because the embeddings do not exist or were never trained, and the second because rules are tied to specific absolute positions rather than relative offsets.
- S5.** A token has T relative positions (one per token, including itself), and overall there are $2T - 1$ distinct distances, needing $2T - 1$ embeddings (APE needs T). Naively adding all T relative embeddings into each token costs T^2 additions. Worse, the sum collapses pairwise information: the token no longer knows which neighbour is at which distance. Relative information is inherently pairwise, so it belongs in the pairwise score e_{ij} .
- S6.** $e_{ij} = (x_i + p_i)W_Q W_K^\top (x_j + p_j)^\top$ expands to word–word + word–position + position–word + position–position. The first is the position-free score, the middle two correlate a word with a position, and the last is a purely positional constant. So PE simply adds terms to e_{ij} . BERT’s visualisation (TUPE paper) shows the two mixed terms are essentially uniform, with no correlation between words and positions: they are a positional artefact of adding APE at the input. This motivates adding position terms to the score directly.
- S7.** Shaw et al. add a learned relative vector to each key, $e_{ij} = x_i W_Q (x_j W_K + p_{ij}^K)^\top$, and to each value, $z_i = \sum_j \alpha_{ij} (x_j W_V + p_{ij}^V)$. The vectors have dimension d (head size), not d_{model} , and are shared across heads but not layers. With $p_{ij} = w_{\text{clip}(j-i, k)}$, all distances beyond $\pm k$ share the same embedding, so only $2k + 1$ vectors exist whatever T is. Clipping does not hurt performance, and with $k = 16$ it beat APE in translation BLEU.
- S8.** Transformer-XL: $e_{ij} = x_i W_Q W_K^\top x_j^\top + x_i W_Q R_{j-i}^\top + u W_K^\top x_j^\top + v W_K^\top R_{j-i}^\top$, with sinusoidal (non-learned) R_{j-i} and learnable global vectors u, v replacing the query-side absolute position. T5: $e_{ij} = x_i W_Q W_K^\top x_j^\top + r_{j-i}$, one learned scalar per (bucketed) distance, shared across layers, clipped at 128. T5 is far cheaper in parameters. Transformer-XL caches hidden states from the previous segment as extra context; absolute positions would collide across segments, so it needed position information that depends only on distance.
- S9.** In “the book”, “read the book”, “you read the book”, ... the distance between “the” and “book” is always 1, so their interaction should be identical. With APE, the vectors become $\text{the} + p_m$ and $\text{book} + p_{m+1}$; since p_m changes with m , the angle (hence $q \cdot k$) between them changes with where the phrase occurs. The fix is to make position act by *rotation*: rotating both vectors by angles proportional to their positions preserves the angle between them up to the relative offset.
- S10.** Let $f_q = x_m W_Q e^{jm\theta}$, $f_k = x_n W_K e^{jn\theta}$. The complex inner product $f_q f_k^H = x_m W_Q (x_n W_K)^H e^{jm\theta} e^{-jn\theta} = x_m W_Q (x_n W_K)^H e^{j(m-n)\theta}$; take the real part for g . Only $m - n$ appears. If $\theta = 0$ the multiplier is 1 for every position, so no positional information is encoded at all.
- S11.** $R_{\Theta, m}^d$ is block-diagonal with 2×2 blocks $R(m\theta_i)$, $\theta_i = 10000^{-2(i-1)/d}$. Then $q_m^\top k_n = x_m^\top W_Q^\top (R_{\Theta, m}^d)^\top R_{\Theta, n}^d W_K x_n$, and block-wise $R(m\theta_i)^\top R(n\theta_i) = R((n - m)\theta_i)$, giving $R_{\Theta, n-m}^d$.

Efficiently: $Rv = v \odot \cos + \text{rot}(v) \odot \sin$, where $\text{rot}(v) = (-v_2, v_1, -v_4, v_3, \dots)$ and the $\cos/\sin$ vectors repeat each $m\theta_i$ twice. That is $\mathcal{O}(d)$ per vector with precomputed tables.

- S12.** Sinusoidal APE is added once, at the input embedding, and mixes into queries, keys and values. RoPE is applied inside every attention layer, only to queries and keys (not values), and is multiplicative (a rotation) rather than additive. The result is a score that depends on relative position only, with no word–position cross terms.
- S13.** ALiBi adds a fixed, non-learned bias $-m \cdot (i - j)$ to each score (no input PE, applied at every layer). Head slopes form a geometric sequence, $m_i = 2^{-8i/n}$, i.e. $1/2^i$ for 8 heads. Because the penalty grows linearly with distance, $e^{-m(i-j)}$ exponentially suppresses far tokens: each head behaves like a soft local window of a different width. At longer test lengths the model still attends mostly locally, exactly as in training, so perplexity stays flat (trained at 512, stable to 16K). Being fixed, it also adds no parameters and is fast.
- S14.** NoPE is a decoder with no explicit PE. The causal mask differs per position (token i sees i tokens), so the model can infer position, e.g. a uniform head gives $z_i = \frac{1}{i} \sum_{j \leq i} v_j$, which encodes i . Findings: perplexity close to PE models (WikiText-103 20.97 vs 19.71 for ALiBi); on downstream length-generalization tasks NoPE had the best mean reciprocal rank (0.69) ahead of T5 (0.55), ALiBi (0.50), RoPE (0.33) and APE (0.22). Theory shows NoPE can represent both absolute and relative PE and in practice learns relative PE.
- S15.** Not solved. ALiBi’s success is measured with perplexity, which rewards local prediction and may hide failures on tasks that need exact long-range reasoning (sorting, addition, reversal). NoPE and T5 do better on such tasks in one study, but no scheme generalises reliably. The Feb 2024 DeepMind study shows Transformers can length-generalise (e.g. on addition) with the right PE and formatting, but not robustly: results vary with seed and data order. It remains an open problem, possibly tied to the architecture itself.

6.5 Part C: Multiple-choice questions

- Q1.** Without positional encoding, self-attention is (a) order-sensitive (b) permutation-equivariant (c) causal (d) linear
- Q2.** In vanilla APE the position vector is combined with the word embedding by (a) multiplication (b) concatenation (c) addition (d) rotation
- Q3.** In sinusoidal PE, dimensions $2k$ and $2k + 1$ (a) use different frequencies (b) share one frequency, sine and cosine (c) are both sines (d) are learned
- Q4.** Which model uses learned APE? (a) T5 (b) LLaMA (c) BERT (d) BLOOM
- Q5.** LLaMA’s position encoding is best classified as (a) absolute learned (b) absolute fixed (c) relative fixed (d) relative learned
- Q6.** T5 is classified as (a) absolute fixed (b) relative learnable (c) relative fixed (d) no PE
- Q7.** In the shift experiment, as the shift k increases, APE model performance (a) is unchanged (b) improves (c) degrades (d) oscillates exactly
- Q8.** A GPT with learned APE and $T = 512$ asked for 1024 tokens fails mainly because (a) attention is quadratic (b) positions 513–1024 have no embeddings (c) the vocabulary is too small (d) softmax overflows
- Q9.** Generalising from 4-digit to 12-digit addition within the context window is an example of (a) extrapolation beyond context (b) length generalization (c) overfitting (d) tokenization
- Q10.** In a sequence of length T , the number of distinct relative positions is (a) T (b) T^2 (c) $2T - 1$ (d) $2T + 1$
- Q11.** The naive RPE implementation needs (a) T additions (b) $T \log T$ (c) T^2 additions (d) $2T - 1$
- Q12.** In the relative-position matrix, the entry for (row “somewhere”, column “something”) is (a) -1 (b) 0 (c) 1 (d) 7
- Q13.** In e_{ij} ’s decomposition, $p_i W_Q W_K^\top p_j^\top$ is (a) word–word (b) word–position (c) position–word (d) position–position
- Q14.** BERT’s visualisation shows the word–position and position–word terms are (a) strongly diagonal (b) nearly uniform (no correlation) (c) negative (d) identical to word–word
- Q15.** In Shaw et al., p_{ij}^K has dimension (a) d_{model} (b) d (head dim) (c) T (d) 1
- Q16.** $\text{clip}(7, 4)$ equals (a) 7 (b) 3 (c) 4 (d) -4
- Q17.** With clipping distance k , the number of relative embeddings is (a) k (b) $2k$ (c) $2k + 1$ (d) $2T - 1$
- Q18.** Shaw’s relative embeddings are shared (a) across layers, not heads (b) across heads, not layers (c) across both (d) across neither
- Q19.** Shaw et al. also add relative information to (a) queries (b) values (c) layer norm (d) the output projection
- Q20.** The clipping distance used in the slide’s MT results table is (a) 4 (b) 8 (c) 16 (d) 128
- Q21.** In Transformer-XL, R_{j-i} is (a) learned (b) sinusoidal (non-learnable) (c) random (d) zero

- Q22.** In Transformer-XL, u and v are (a) fixed scalars (b) learnable vectors (c) positions (d) values
- Q23.** T5 bias adds to the score (a) a learnable scalar r_{j-i} (b) a vector p_{ij} (c) a rotation (d) $-m(i-j)$
- Q24.** T5's relative biases are (a) per layer (b) shared across layers (c) per token (d) random
- Q25.** RoPE aims for $\langle f_q(x_i, i), f_k(x_j, j) \rangle$ to depend on (a) i only (b) $i+j$ (c) $j-i$ (d) ij
- Q26.** The matrix $\begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix}$ is a (a) reflection (b) rotation (c) projection (d) shear
- Q27.** $R(a)^\top R(b)$ equals (a) $R(a+b)$ (b) $R(b-a)$ (c) I (d) $R(ab)$
- Q28.** In RoPE, θ must be non-zero because otherwise (a) gradients vanish (b) no position information is encoded (c) norms explode (d) softmax fails
- Q29.** RoPE's frequencies are $\theta_i =$ (a) i/d (b) $10000^{-2(i-1)/d}$ (c) 2^{-i} (d) learned
- Q30.** RoPE is applied to (a) values only (b) queries and keys (c) input embeddings once (d) the output layer
- Q31.** Integration type of RoPE in the summary table (a) add (b) multiply (c) concatenate (d) none
- Q32.** In the ALiBi extrapolation experiment the model was trained with $T =$ (a) 128 (b) 512 (c) 2048 (d) 16000
- Q33.** Which PE's perplexity stays flat up to 16K tokens? (a) sinusoidal (b) rotary (c) T5 bias (d) ALiBi
- Q34.** Increasing training context to 1024 helped most (a) sinusoidal (b) rotary (c) T5 bias (d) learned APE
- Q35.** ALiBi slopes for 8 heads are (a) $1, 2, \dots, 8$ (b) $1/2^i$ (c) learned (d) all equal
- Q36.** The ALiBi bias for query 5 and key 2 (slope m) is (a) $3m$ (b) $-3m$ (c) $-2m$ (d) 0
- Q37.** ALiBi extrapolates well mainly because it (a) memorises long sequences (b) acts like a soft local window (c) uses learned embeddings (d) removes the softmax
- Q38.** NoPE relies on which component to recover positional information? (a) layer norm (b) the causal mask (c) dropout (d) the tokenizer
- Q39.** In the downstream length-generalization study, the best mean reciprocal rank was achieved by (a) APE (b) RoPE (c) ALiBi (d) NoPE
- Q40.** Theoretically NoPE (a) cannot represent position (b) can represent both absolute and relative PE, and uses relative in practice (c) only absolute (d) only works in encoders
- Q41.** Every RPE in the summary table injects position at (a) the initial layer only (b) every layer (c) the output only (d) no layer
- Q42.** The Feb 2024 DeepMind study concluded Transformers (a) cannot length-generalise (b) can length-generalise but not robustly (c) generalise perfectly (d) need no PE

6.6 Part C: Answer key with explanations

- Q1.** (b) Permuting the inputs permutes the outputs identically.

- Q2. (c) $h_i = x_i + p_i$.
- Q3. (b) Pairs share ω_k : one sine, one cosine.
- Q4. (c) BERT, GPT-x, BART and OPT use learned APE.
- Q5. (c) RoPE is a fixed formula that yields relative dependence.
- Q6. (b) Learned scalar relative biases.
- Q7. (c) So APE did not truly learn relative positions.
- Q8. (b) The learned table has only 512 rows.
- Q9. (b) Longer test instances within the context window.
- Q10. (c) From $-(T - 1)$ to $T - 1$.
- Q11. (c) T additions per token $\times T$ tokens.
- Q12. (c) $j - i = 1 - 0 = 1$.
- Q13. (d) Both factors are positions.
- Q14. (b) This is the positional artefact.
- Q15. (b) Head dimension, versus d_{model} for APE.
- Q16. (c) $\max(-4, \min(7, 4)) = 4$.
- Q17. (c) $w_{-k}, \dots, w_k$.
- Q18. (b) Stated in the slides.
- Q19. (b) $z_i = \sum_j \alpha_{ij}(x_j W_V + p_{ij}^V)$.
- Q20. (c) $k = 16$ gave better BLEU than APE.
- Q21. (b) Computed with the sinusoid formula.
- Q22. (b) They replace the query-side absolute position term.
- Q23. (a) $e_{ij} = x_i W_Q W_K^\top x_j^\top + r_{j-i}$.
- Q24. (b) This keeps the parameter count small.
- Q25. (c) $g(x_i, x_j, j - i)$.
- Q26. (b) Determinant 1, orthogonal.
- Q27. (b) $R(-a)R(b) = R(b - a)$.
- Q28. (b) $e^{j \cdot 0} = 1$ for all positions.
- Q29. (b) The same geometric frequencies as sinusoidal PE.
- Q30. (b) Values are untouched; applied at every layer.
- Q31. (b) A rotation is multiplicative.
- Q32. (b) Tested with T_{valid} from 512 to 16K.
- Q33. (d) The others degrade.
- Q34. (c) This motivated modifying T5 bias, which led to ALiBi.
- Q35. (b) A geometric progression $1/2, \dots, 1/256$.

- Q36. (b)** $-m(i - j) = -3m$.
- Q37. (b)** Distant keys are exponentially down-weighted.
- Q38. (b)** Each position sees a different number of keys.
- Q39. (d)** NoPE 0.69 > T5 0.55 > ALiBi 0.50 > RoPE 0.33 > APE 0.22.
- Q40. (b)** As shown theoretically in the study.
- Q41. (b)** APE injects once at the input; RPE at every layer.
- Q42. (b)** “Transformers Can Achieve Length Generalization But Not Robustly”.